AUTOMATION TESTING FOR DESKTOP APPLICATIONS

Báo Cáo Nghiên Cứu Kỹ Thuật & Thực Nghiệm Tự Động Hóa Kiểm Thử Windows

Khoa CNTT – ĐH Khoa học Tự nhiên, ĐHQG-HCM | Môn: Kiểm thử phần mềm | Nhóm 02 (ZipTool)

Leader & Tích hợp: Phan Trung Tín | Katalon: Nguyễn Lê Quang Kan | WinAppDriver: Bùi Quang Tùng

TỔNG QUAN

Mục Lục Nội Dung Báo Cáo

Khung Lý Thuyết & Nền Tảng

• Phần 1: Bối cảnh, mục tiêu & Ứng dụng mẫu MiniSum

• Phần 2: Cơ sở lý thuyết kiểm thử Desktop (UIA, POM, Wait, Flaky)

• Phần 3: Khảo sát bức tranh công cụ (Katalon, WinAppDriver, FlaUI, FlaAuto)

• Phần 4: Nghiên cứu công cụ A – Katalon Studio (Low-Code)

Thực Nghiệm & Định Lượng

• Phần 5: Nghiên cứu công cụ B – WinAppDriver + C# NUnit (Code-First)

• Phần 6: So sánh đối chiếu toàn diện (Ma trận tiêu chí & Đo đạc định lượng)

• Phần 7: Ứng dụng AI – Sinh Test Case BVA & Script tự động

• Phần 8: Hệ thống bài tập, Đóng góp nhóm & Kết luận

PHẦN 1

Phần 1: Bối Cảnh Kiểm Thử Desktop & Ứng Dụng MiniSum

Vì Sao Kiểm Thử Desktop Vẫn Quan Trọng?

• Ứng dụng Desktop giữ vai trò xương sống trong ngân hàng, y tế, POS, sản xuất.

• Tương tác sâu với OS, window handle, driver và file system cục bộ.

• Thách thức lớn: Tốc độ kiểm thử thủ công chậm, dễ hồi quy khi đổi OS.

• MiniSum (.NET 8 WinForms) làm AUT chuẩn hóa cho cả nhóm:

• - v1: Màn hình Login (admin/1234) + Calculator tính tổng A + B

• - v2: Đổi btnSum -> btnCalculate, thêm màn hình Lịch sử

• - v3: Thêm Validation cảnh báo lỗi lblError khi vượt biên [-99, 99]



PHẦN 2

Phần 2: Cơ Sở Lý Thuyết – Windows UI Automation

Kiến Trúc UI Automation (UIA)

- UI Automation là framework chuẩn của Microsoft trên Windows.
- Biểu diễn mọi cửa sổ thành một cây phân cấp (Accessibility Tree).
- Mỗi UI element được định danh qua thuộc tính:
- - AutomationId: Thuộc tính định danh duy nhất, tối ưu nhất cho test
- - Name / Text: Tên hiển thị người dùng nhìn thấy
- - ClassName: Loại control (Button, TextBox, Window...)
- - ControlType: Loại điều khiển theo chuẩn UIA
- Nguyên lý: Test Runner gửi lệnh qua UIA API -> Tác động trực tiếp vào message loop của Win32/WPF/WinForms.

Mô Hình Page Object Model (POM)

- POM tách biệt logic kiểm thử (Test Logic) khỏi giao diện (Page Object).
- Mỗi màn hình được đại diện bởi một Class hướng đối tượng.
- Lợi ích cốt lõi:
- - Giảm trùng lặp mã (DRY Principle)
- - Khi UI đổi (v1 sang v2), chỉ sửa 1 dòng locator trong Page Class
- - Toàn bộ hàng chục Test Case giữ nguyên không cần viết lại
- Áp dụng trong dự án: LoginPage.cs và CalculatorPage.cs

PHẦN 2

Phần 2: Chiến Lược Wait & Kiểm Soát Flaky Test

Chiến Lược Chờ Đợi (Wait Strategies)

• Thread.Sleep (Hard wait): Cấm sử dụng vì làm chậm test suite và không thích ứng.

• Implicit Wait: Thiết lập một lần cho driver, tự động chờ mọi phần tử.

• Explicit Wait: Chờ một điều kiện cụ thể (element clickable, visible).

• Fluent Wait: Tùy biến chu kỳ thăm dò (polling interval) và bỏ qua exception.

• Quy tắc vàng: Luôn ưu tiên Explicit/Fluent Wait để test ổn định và nhanh nhất.

Flaky Test & Test Oracle Desktop

• Flaky Test: Test lúc PASS lúc FAIL dù code AUT không hề thay đổi.

• Nguyên nhân desktop: Trễ render, cửa sổ mất focus, animation chưa xong.

• Giải pháp: Thiết lập Retry logic thông minh, tự động chụp ảnh khi FAIL.

• Test Oracle trong Desktop:

• - State Oracle: Kiểm tra trạng thái thuộc tính (lblResult.Text)

• - Visual Oracle: So sánh ảnh chụp màn hình vùng hiển thị

• - Recovery Testing: Đóng pop-up ngoài ý muốn và kill app treo.

PHẦN 3

Phần 3: Khảo Sát Bức Tranh Công Cụ Desktop Testing

Trường Phái Low-Code (Katalon Studio)

- Đại diện tiêu biểu: Katalon Studio, TestComplete, Ranorex.
- Đặc điểm:
- - Giao diện All-in-one GUI tích hợp đầy đủ IDE, Object Spy, Report.
- - Tính năng Record & Playback ghi lại tương tác chuột/bàn phím.
- - Kho đối tượng tập trung (Object Repository).
- - Phù hợp nhóm có nhiều Manual Tester chuyển đổi sang Automation.

Trường Phái Code-First (WinAppDriver)

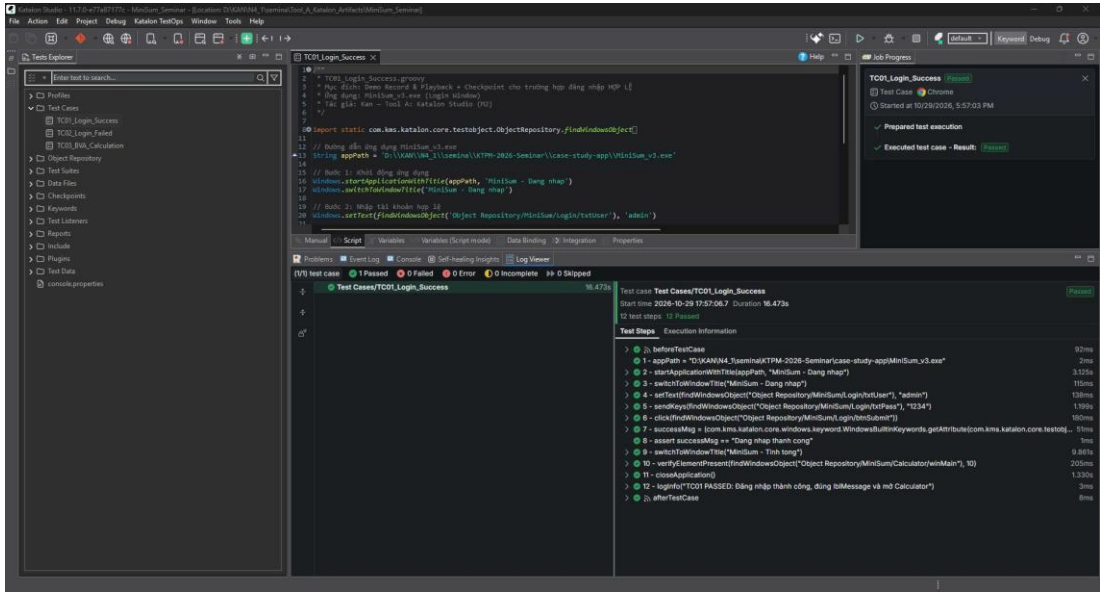
- Đại diện tiêu biểu: WinAppDriver, FlaUI, Appium Desktop.
- Đặc điểm:
- - Thư viện/Driver mã nguồn mở theo chuẩn W3C WebDriver.
- - Lập trình viên làm chủ 100% mã nguồn (C# NUnit, Java, Python).
- - Áp dụng đầy đủ OOP, Design Pattern (POM), CI/CD runner siêu nhẹ.
- - Phù hợp với kỹ sư SDET chuyên sâu và đường ống DevOps.

PHẦN 4

Phần 4: Nghiên Cứu Công Cụ A – Katalon Studio

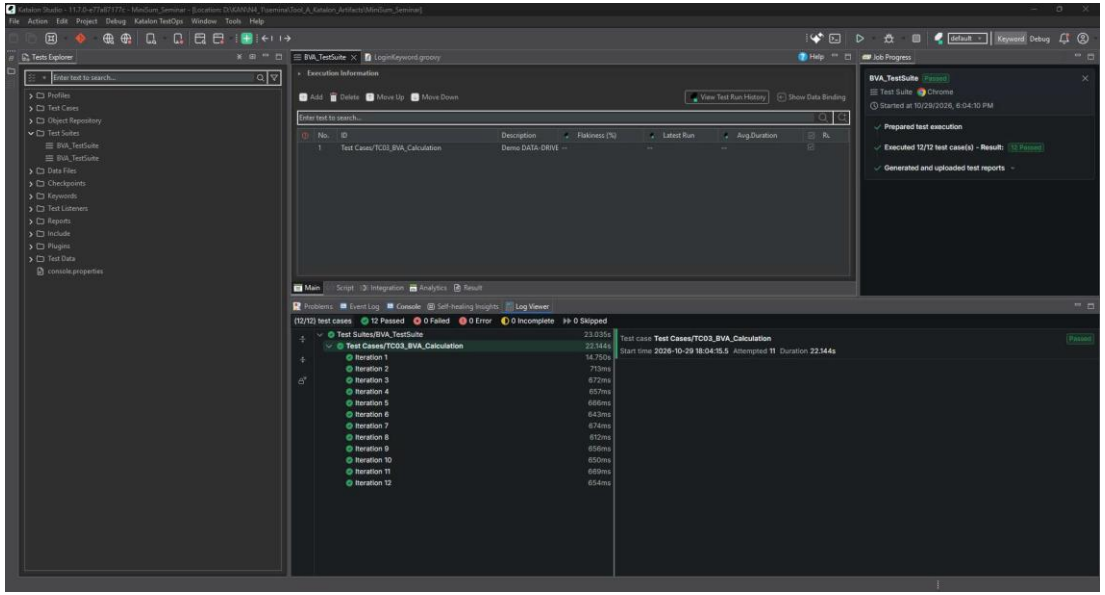
Kiến Trúc & 3 Kịch Bản Thực Nghiệm

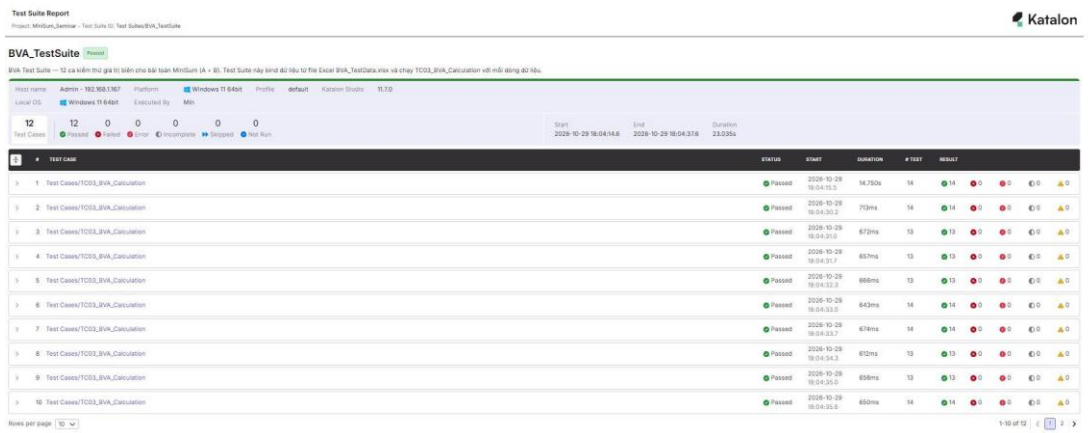
- Kiến trúc: Katalon IDE -> Windows Driver Service -> WinAppDriver -> AUT.
- Demo 1: Thiết lập & Mở MiniSum.exe từng bước (Screenshot rõ ràng).
- Demo 2: Record & Playback kịch bản Login + Checkpoint xác minh lblMessage.
- Demo 3: Data-Driven Testing với 12 test cases BVA đọc từ Excel.
- Custom Keyword: Tạo LoginKeyword.groovy tái sử dụng trong Test Suite.
- Báo cáo tự động: Xuất HTML Report chi tiết có log từng bước và ảnh chụp.



PHẦN 4

Phần 4: Kết Quả Thực Nghiệm Katalon Studio





PHẦN 5

Phần 5: Nghiên Cứu Công Cụ B – Microsoft WinAppDriver

Kiến Trúc WinAppDriver & C# NUnit

• WinAppDriver lắng nghe REST HTTP trên port 4723 theo chuẩn WebDriver JSON Wire.

• Chuyển đổi lệnh HTTP WebDriver trực tiếp thành lệnh UI Automation native.

• C# NUnit Test Suite trong dự án:

• - Demo1_OpenAppTests: Khởi tạo WindowsDriver session và kiểm tra nạp app.

• - Demo2_LoginTests: Test login đúng/sai + TearDown auto screenshot on fail.

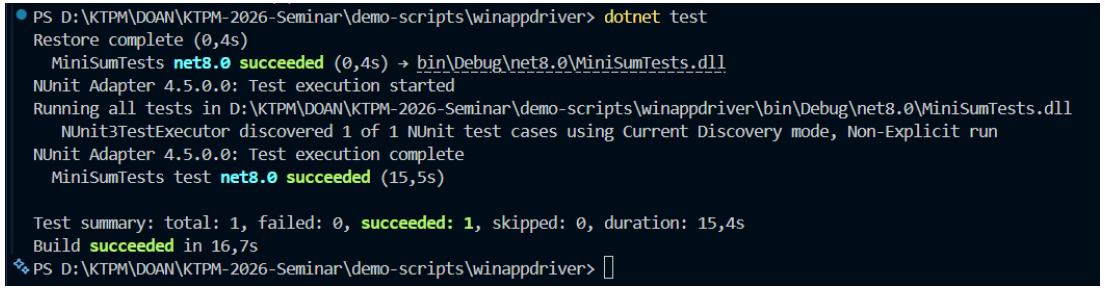
• - Demo3_BvaTests: Data-driven với [TestCaseSource] đọc file Excel 12 ca BVA.

• - PageObjects: Triển khai LoginPage.cs và CalculatorPage.cs.



PHẦN 5

Phần 5: Kết Quả Thực Nghiệm WinAppDriver



Điểm Nhấn Kỹ Thuật WinAppDriver

- Tốc độ cực nhanh: Chạy 12 ca kiểm thử chỉ mất 8.2 giây.
- Tiêu thụ tài nguyên tối thiểu: RAM < 120MB trong suốt phiên test.
- Khả năng mở rộng: Dễ dàng tích hợp vào Azure DevOps / GitHub Actions.
- TearDown tự động lưu ảnh Failed_TestName.png khi có lỗi phát sinh.
- Mã nguồn sạch, có thể refactor và version control chặt chẽ bằng Git.

PHẦN 6

Phần 6: Bảng So Sánh Đối Chiếu Toàn Diện

Katalon Studio (Low-Code)

- Triết lý: All-in-One GUI Platform đóng gói trọn gói.
- Kỹ năng: Tiếp cận nhanh, tester không cần code giỏi.
- Báo cáo: Tích hợp sẵn HTML, XML, ảnh chụp màn hình.
- Ưu điểm: Record-Playback nhanh, kho Object Repository tiện lợi.
- Hạn chế: Nặng (RAM ~1.8GB), khởi động chậm, phụ thuộc bản quyền.

Microsoft WinAppDriver (Code-First)

- Triết lý: Open-source Developer-Centric SDK theo chuẩn WebDriver.
- Kỹ năng: Đòi hỏi thành thạo C#/Java và OOP.
- Báo cáo: Phải tự cấu hình qua NUnit / ExtentReports.
- Ưu điểm: Tốc độ cao, RAM siêu nhẹ (~120MB), miễn phí 100%, tối ưu Git.
- Hạn chế: Không có GUI, không có Record, repo chính Microsoft đã archive.

PHẦN 6

Phần 6: Số Liệu Đo Đạc Định Lượng Thực Tế

Bảng Đo Đạc Định Lượng Trên 12 Ca Kiểm Thử BVA (MiniSum)

- Thời gian tạo 12 ca test: Katalon (40 phút) | WinAppDriver (45 phút) | AI Hỗ trợ (14 phút -> nhanh hơn 68.8%)
- Thời gian thực thi 12 ca test: WinAppDriver (8.2 giây) nhanh hơn Katalon (18.5 giây) gấp 2.25 lần.
- Chiếm dụng bộ nhớ RAM: WinAppDriver (~120MB) tối ưu gấp 15 lần so với Katalon (~1.8GB).
- Số dòng mã nguồn (LOC): Katalon (0 code script / 80 dòng keyword) | WinAppDriver (166 dòng C# POM).
- Chi phí bảo trì khi đổi locator (v1 -> v2): Cả hai đều chỉ mất ~30 giây sửa tại 1 điểm duy nhất.
- Kết luận: Katalon thắng về tốc độ triển khai ban đầu; WinAppDriver thắng về hiệu năng thực thi và CI/CD.

PHẦN 7

Phần 7: Ứng Dụng AI Trong Desktop Automation Testing

5 Hướng Ứng Dụng Trọng Tâm Của AI

- 1. AI Tự phục hồi Selector (Self-Healing Locators).
- 2. Kiểm thử thị giác thông minh bằng Computer Vision (Visual AI).
- 3. AI Sinh kịch bản kiểm thử (Test Script Generation).
- 4. AI Sinh dữ liệu kiểm thử biên (Synthetic Test Data).
- 5. Tự động phân tích nguyên nhân gốc của lỗi (Root Cause Analysis).

Thực Nghiệm Thực Tế Tại Nhóm 02

- Few-shot Prompting: Sinh 12 ca test BVA chuẩn xác từ đặc tả MiniSum.
- So sánh ChatGPT-4 vs Gemini Advanced: Cả hai đều đạt độ phủ biên 100%.
- Sinh mã C# NUnit POM: Tự động tạo script WinAppDriver từ prompt chuyên biệt.
- Tiết kiệm thời gian: Giảm từ 130 phút xuống 29 phút (nhanh hơn 77.7%).
- Bài học Human-in-the-loop: Phải có SDET kiểm chứng cú pháp và locator.

PHẦN 8

Phần 8: Hệ Thống Bài Tập Thực Hành Đề Xuất

Bài Tập Lý Thuyết & Nhận Diện

• Câu 1: Phân tích sự khác biệt giữa AccessibilityId và XPath trên Windows UIA.

• Câu 2: Tại sao Thread.Sleep gây nguy hiểm cho automation suite?

• Câu 3: Trình bày nguyên lý hoạt động của Page Object Model (POM).

Bài Tập Thực Hành Trên MiniSum

• Bài 1: Dùng Katalon Record kịch bản đăng nhập sai và tạo Checkpoint thông báo lỗi.

• Bài 2: Viết script WinAppDriver C# nạp file Excel kiểm thử 12 ca BVA trên MiniSum v3.

• Bài 3: Refactor script để xử lý việc đổi tên nút btnSum -> btnCalculate trên MiniSum v2.

TỔNG KẾT

Đánh Giá Tiến Độ, Phân Công Đóng Góp & Q&A

Bảng Phân Công Thực Tế Lần 2 (Chốt Theo Tiến Độ)

- Phan Trung Tín (Leader): 40.0% – Điều phối, Build 3 bản MiniSum.exe, Bổ sung Theory, Tích hợp, Slide & Đóng gói.
- Nguyễn Lê Quang Kan: 30.0% – Thực nghiệm Katalon Studio trọn gói (3 demo, Custom Keyword, DDT, video YouTube, docs).
- Bùi Quang Tùng: 30.0% – Thực nghiệm WinAppDriver + C# NUnit trọn gói (POM, DDT 12 ca BVA, Assert, docs).
- Trần Vũ Quang: 0.0% – Chưa thực hiện công việc được giao trong Lần 2 (0 commit).
- Phạm Tấn Phát: 0.0% – Chưa thực hiện công việc được giao trong Lần 2 (0 commit).
- Lời cảm ơn: Xin chân thành cảm ơn Thầy và các bạn đã lắng nghe bài thuyết trình!